



ALLAMA IQBAL OPEN UNIVERSITY
MAIN CAMPUS H-8, ISLAMABAD

PROJECT REPORT

Project Title

Real Time Device to Device Wireless Communication System Using ESP-NOW Protocol

A two node wireless embedded system that combines an ultrasonic sensor and a motion sensor on a Sender board, and communicates detection status to a Receiver board directly through the ESP-NOW protocol, without using Wi-Fi routers, internet, or cloud services.

Name: Abdul Rehman

Registration Number: 0000674917

Program: BSCS

Project Type: Hardware and Software Based Embedded IoT System

Supervisor: Dr. Israr Akhter (Slovak Technical University Bratislava)

Table of Contents

1	Project Abstract	3
2	Introduction	3
2.1	Background	3
2.2	Problem Statement	4
2.3	Objectives	4
2.4	Scope	4
3	Literature Review	4
4	Proposed System.....	5
4.1	Proposed Model	5
4.2	System Background (ESP-NOW Communication).....	6
4.3	Hardware Components.....	7
4.4	Hardware Diagram	7
4.5	Circuit Connections.....	8
4.6	OSI Model Mapping of the Proposed System	9
4.7	Justification for Selecting the ESP-NOW Protocol	10
4.8	Protocols Required If Cloud or Internet Connectivity Is Added	12
5	Methodology.....	12
5.1	Working Principle	12
5.2	Flowchart.....	14
5.3	Algorithm	14
5.3.1	Sender Node Algorithm	14
5.3.2	Receiver Node Algorithm	15
5.4	Functionalities.....	16
6	Results and Discussion	17
7	Conclusion.....	21
7.1	Project Summary	21
7.2	Achievements	21
7.3	Advantages	21
7.4	Limitations.....	22
8	Future Work	22
9	References	22
10	Appendix	23
10.1	Sender.ino	23
10.2	Receiver.ino	31
10.3	read_log.ino	36

1 Project Abstract

This project presents a low cost, wireless, dual sensor detection and alert system built using two ESP8266 NodeMCU boards that communicate directly through the ESP-NOW protocol, without relying on any WiFi router, internet connection, or cloud service. The core problem addressed is that a single sensor, whether an ultrasonic distance sensor or a passive infrared motion sensor, can produce unreliable or false detections on its own. To solve this, the Sender node combines readings from an HC-SR04 ultrasonic sensor and an SR505 PIR motion sensor, and only treats an event as significant when both sensors detect something together. Before anything is transmitted, the Sender runs a one second stability check to confirm the object stays at roughly the same distance, and once a message has been sent it enforces a three second cooldown before it will consider sending again, which together prevent false triggers and message flooding. The Receiver node, carrying only two LEDs, stays silent and shows a green safe status by default, and only lights its red LED and prints a line to the Serial Monitor when a confirmed combined detection actually arrives; every such line is also appended to a notepad style log file stored on the Receiver's own flash memory using LittleFS, so a permanent record builds up automatically. The system was implemented on a breadboard, tested using the Arduino Serial Monitor on both boards, and verified to correctly gate transmissions on combined, confirmed detections at a fixed 20 centimeter distance threshold. The results confirm that combining two sensing modalities with a confirmation and cooldown mechanism, over a direct peer to peer wireless link, produces a dependable, internet free monitoring solution suitable for small scale security and environment monitoring tasks. The project demonstrates practical applications such as entry point monitoring, restricted area alerts, and low cost IoT education, while also highlighting clear directions for future improvement such as multi sender support, real time clock timestamps, and mobile notifications.

2 Introduction

2.1 Background

Embedded wireless sensor systems are widely used today to monitor spaces, detect movement, and trigger alerts without needing a person to be physically present. Traditional systems often depend on a WiFi router and sometimes a cloud server to move data from a sensing device to a display or notification device. This dependency introduces recurring costs, and it also introduces a single point of failure, since the whole system stops working the moment the internet connection or the cloud service becomes unavailable. Espressif Systems, the manufacturer of the ESP8266 and ESP32 microcontroller families, addressed part of this problem by introducing ESP-NOW, a connectionless wireless protocol that allows two compatible boards to exchange small data packets directly using only their MAC addresses, completely bypassing any router or internet connection.

This project uses two ESP8266 NodeMCU development boards along with an ultrasonic distance sensor and a passive infrared motion sensor to build a compact, dependency free wireless detection and alert system, where the communication between the Sender and the Receiver happens entirely through ESP-NOW.

2.2 Problem Statement

In many practical monitoring situations, relying on a single sensor is not always enough to correctly judge whether something meaningful is happening in an area. A motion sensor working alone can trigger falsely due to heat sources, air movement, or electrical noise, while an ultrasonic sensor working alone can trigger due to a stationary object that has nothing to do with real activity. In addition, many existing wireless alert designs still depend on a WiFi router or a cloud dashboard, which adds cost, setup complexity, and a dependency on internet availability. This project addresses both weaknesses together, by combining two different sensing techniques on one Sender device, and by wirelessly forwarding the combined detection result to a separate Receiver device using a direct, router free, internet free communication method.

2.3 Objectives

The main objective of this project is to combine an ultrasonic distance sensor and a passive infrared motion sensor on a single Sender board, so that both sensors work together and produce a more reliable detection outcome than either sensor could produce alone.

Another objective is to establish a direct wireless connection between two ESP8266 NodeMCU boards using the ESP-NOW protocol, so that detection information can travel from the Sender to the Receiver without needing a WiFi router, an internet connection, or any third party server.

A further objective is to represent the current detection status through simple LED indicators on both the Sender and the Receiver side, so that the system status can be understood instantly, without needing a computer, a mobile application, or any additional display hardware.

The project also aims to demonstrate a practical, low cost example of peer to peer wireless embedded communication that can later be expanded into a more complete security or environment monitoring system.

2.4 Scope

The scope of this project covers the design, wiring, programming, and testing of exactly two ESP8266 NodeMCU boards operating as a Sender and a Receiver. The Sender is limited to one ultrasonic sensor and one PIR motion sensor, and the detection distance threshold is fixed at 20 centimeters in the current implementation. The project focuses on local, direct device to device communication, and it does not include internet connectivity, a mobile application, or cloud based data storage, since the entire purpose of using ESP-NOW here is to avoid those dependencies. The system is intended as an academic demonstration and a foundation for future extension, rather than a finished commercial security product.

3 Literature Review

Combining an ultrasonic sensor with a passive infrared sensor to reduce false detections is not a new idea. Bai et al. designed an embedded surveillance system that paired ultrasonic signal coding with multiple pyroelectric infrared sensors to detect an intruder inside a home or a storehouse, and showed that combining the two sensing methods reduced the miss rate that a single ultrasonic sensor would otherwise suffer from due to environmental interference. This earlier work supports the central design choice of the present project, which similarly fuses ultrasonic and PIR readings on the Sender node, although the present project uses a simpler

distance threshold rule rather than signal coding, since the goal here is a low cost student project rather than a research grade anti interference system.

On the ESP8266 platform specifically, Kumar Aditya published an open hardware project on Hackster.io that used a NodeMCU with a PIR sensor to build a smart home security notifier, sending email and mobile notifications whenever motion was detected. That project demonstrates the ESP8266 as a capable low cost platform for motion based alerting, but it relies on a WiFi network and an internet connected notification service, which is exactly the dependency that the present project avoids by using ESP-NOW instead.

Closer to the communication side of this project, a residential monitoring study proposed combining ultrasonic sensors, multiple PIR sensors, flame sensors, and gas sensors on ESP32 boards connected through an ESP-MESH network, and used the collected sensor data to detect intrusions and other events across several houses. This confirms that combining several sensing modalities on ESP based boards is a recognized and effective approach for monitoring applications, although that system still forwarded data to an online server for final decision making, unlike the fully local, cloud free approach used in this project.

Several independent studies have also examined the ESP-NOW protocol itself rather than a specific sensing application. One performance study proposed an adaptable mesh topology built on ESP-NOW and reported that it achieved a substantially longer communication distance than Bluetooth Low Energy while only doubling power consumption, confirming that ESP-NOW is a practical choice for extending sensor coverage without a router. A separate field experiment study analyzed ESP-NOW behavior in outdoor environments and confirmed that the protocol operates at the WiFi data link layer using action frames, which keeps its latency and energy consumption low compared to protocols that rely on a full WiFi association process. These studies collectively support the choice of ESP-NOW as the communication backbone for the present project, since they confirm it is both fast enough and power efficient enough for small, periodic status updates such as the one used between the Sender and Receiver nodes here.

Compared to this existing work, the present project's contribution is a simple, easily reproducible student level combination of the two ideas, dual sensor fusion for more reliable detection, and pure ESP-NOW communication for a completely router free and internet free alert path, expressed entirely through low cost LEDs rather than a network dashboard or mobile notification service.

4 Proposed System

4.1 Proposed Model

The proposed model is organized as two independent nodes connected only through a wireless ESP-NOW link, as shown in the block diagram below. The Sender node collects readings from both sensors, and only when both agree that something is present does it start a one second confirmation check; if the object stays at the same distance for that full second, the Sender transmits a status packet and then enforces a three second cooldown before it will consider sending again. The Receiver node has no sensors of its own and exists purely to receive that

packet, light its red LED, print a line to the Serial Monitor, and append the same line to a log file stored on its own flash memory, falling back to its green LED whenever no confirmed packet has arrived recently.

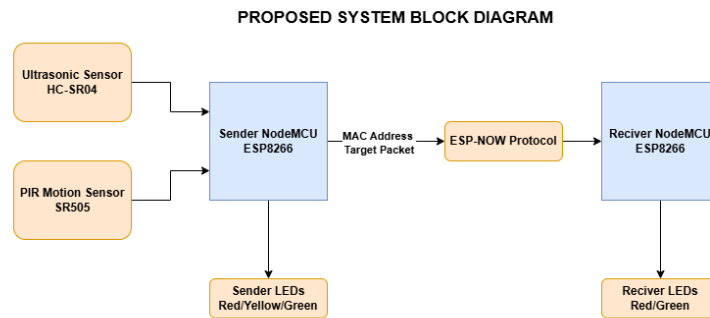


Figure 1: Proposed system block diagram showing the Sender sensing and logic stage, the ESP-NOW Protocol, and the Receiver display stage

4.2 System Background (ESP-NOW Communication)

The proposed system is based on the ESP-NOW wireless communication protocol, which enables direct device-to-device communication between two ESP8266 NodeMCU development boards. Unlike conventional Internet of Things (IoT) systems, the proposed system does not require a Wi-Fi router, internet connection, cloud platform, or external server for communication. Instead, both NodeMCU boards exchange data directly using their unique Media Access Control (MAC) addresses.

ESP-NOW is a proprietary communication protocol developed by Espressif Systems for ESP8266 and ESP32 microcontrollers. The protocol allows multiple ESP devices to communicate with each other through peer-to-peer wireless communication while maintaining low latency and low power consumption. Since no network association or internet connectivity is required, ESP-NOW provides a fast, lightweight, and reliable communication method for embedded systems.

In the proposed project, two ESP8266 NodeMCU boards are used. The first NodeMCU functions as the Sender, while the second operates as the Receiver. The sender continuously monitors the HC-SR04 ultrasonic sensor and the SR505 Passive Infrared (PIR) motion sensor. Based on the sensor readings, the sender determines whether an object, motion, or both have been detected. The processed detection information is then transmitted wirelessly to the receiver using the ESP-NOW protocol.

The receiver NodeMCU listens continuously for incoming ESP-NOW packets. Whenever a valid packet is received, the receiver immediately updates its LED indicators according to the received detection status. A red LED indicates that a detection event has occurred, whereas a green LED indicates that no detection is present or that communication has timed out. This direct communication process enables real-time monitoring without relying on any external networking infrastructure.

The proposed communication architecture provides several advantages over traditional Wi-Fi-based IoT systems. Since communication takes place directly between the sender and receiver, the system offers lower communication delay, reduced power consumption, improved reliability, and simplified deployment. Furthermore, eliminating cloud services and internet

connectivity enhances privacy and allows the system to operate in environments where network access is unavailable.

For these reasons, ESP-NOW was selected as the communication technology for the proposed wireless monitoring system. Its ability to provide fast, reliable, and router-free communication makes it highly suitable for small-scale security, monitoring, and embedded IoT applications.

4.3 Hardware Components

The table below lists every physical component used to build both nodes of the system, along with the quantity used and the role each component plays.

Table 1: Hardware Components used in the Sender and Receiver Sets

Component	Quantity	Used On	Purpose
NodeMCU ESP8266	2	Sender and Receiver	Main microcontroller board with built in Wi-Fi, used for sensor logic and ESP-NOW communication
Ultrasonic Sensor HC-SR04	1	Sender	Measures distance to a nearby object using sound wave reflection, range 2 cm to 400 cm
PIR Motion Sensor SR505	1	Sender	Detects movement of a warm body such as a person passing nearby
LED (Red, Yellow, Green)	3	Sender	Visually shows which sensor combination triggered the detection
LED (Red, Green)	2	Receiver	Visually shows whether the Sender is currently reporting a detection
220 Ω and 230 Ω Resistor	5	Sender and Receiver	Current limiting resistor placed before every LED to protect it from overcurrent
Jumper Wires	20	Sender and Receiver	Makes all electrical connections between the board and its components
Breadboard	4		
Micro USB Cable	2	Sender and Receiver	Powers each NodeMCU board and allows code upload from a computer

4.4 Hardware Diagram

The photographs below show the actual assembled hardware for both nodes on a breadboard, corresponding exactly to the pin connections described in the next section.

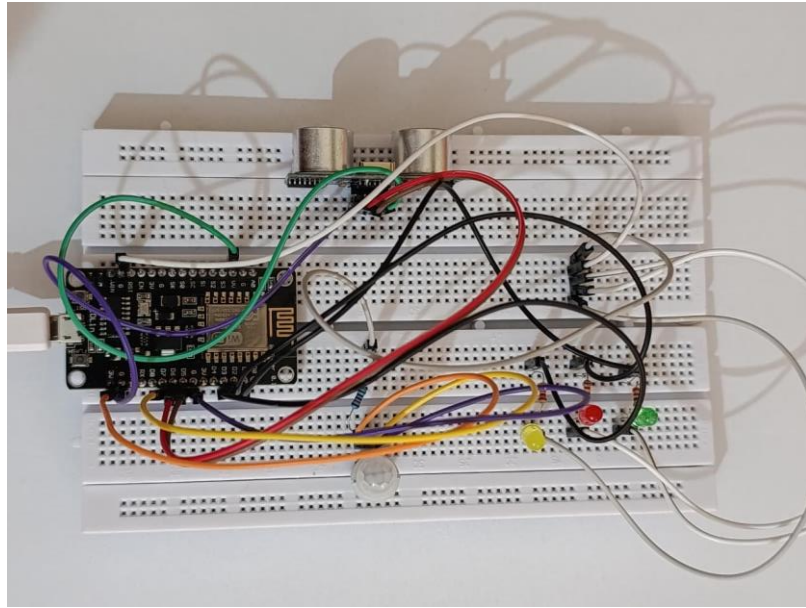


Figure 2: Sender node hardware assembled on breadboard, showing the ultrasonic sensor, the PIR motion sensor, the NodeMCU ESP8266, and the three status LEDs with their current limiting resistors

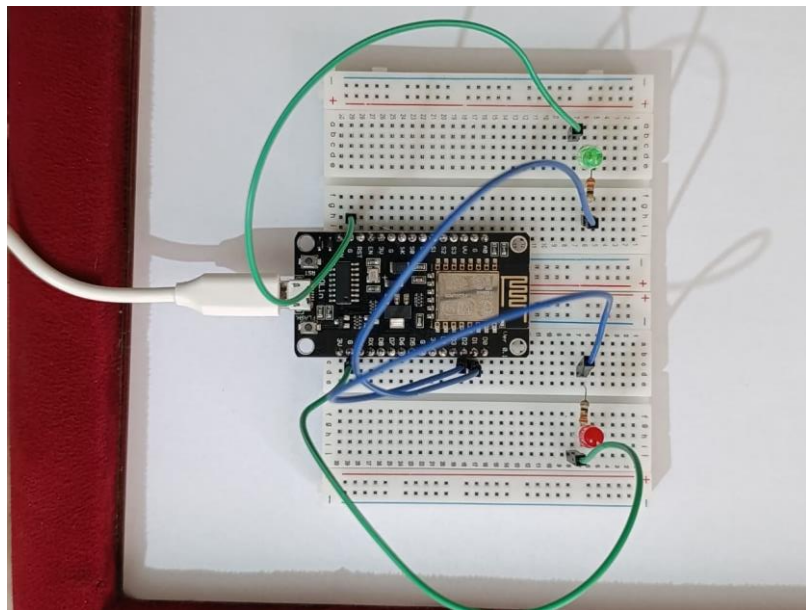


Figure 3: Receiver node hardware assembled on breadboard, showing the NodeMCU ESP8266 and its two status LEDs with current limiting resistors

4.5 Circuit Connections

Table 2: Sender NodeMCU pin connections

NodeMCU Pin	Connected to	Notes
D1	220 Ω resistor to Green LED anode	Green LED cathode connects to common ground
D2	220 Ω resistor to Red LED anode	Red LED cathode connects to common ground
D3	220 Ω resistor to Yellow LED anode	Yellow LED cathode connects to common ground

D5	HC-SR04 TRIG pin	Sends the 10 microsecond trigger pulse to start a measurement
D6	HC-SR04 ECHO pin	Reads the return pulse duration used to calculate distance
D7	SR505 PIR sensor OUT pin	Reads HIGH when motion is detected nearby
VV	HC-SR04 VCC pin	Ultrasonic sensor is powered from the 5V USB line for stable readings
3V	SR505 PIR sensor + pin	PIR sensor is powered from the onboard 3.3V regulator
GND	HC-SR04 GND, PIR GND -, and all three LED cathodes	Common ground shared by every component on the Sender board

Table 3: Receiver NodeMCU pin connections

NodeMCU Pin	Connected to	Notes
D1	220 Ω resistor to Green LED anode	Green LED cathode connects to ground
D2	220 Ω resistor to Red LED anode	Red LED cathode connects to ground
GND	Green LED cathode and Red LED cathode	Common ground shared by both Receiver LEDs

No sensors are attached to the Receiver board. Its only job is to run the ESP8266 Wi-Fi radio in station mode, listen for ESP-NOW packets coming from the Sender, and reflect the received status using its two LEDs.

4.6 OSI Model Mapping of the Proposed System

The working of the proposed system can also be examined through the OSI reference model, which divides network communication into seven layers, from the physical transmission of signals up to the meaning an application gives to that data. A conventional Wi-Fi router based system normally makes use of all seven layers, since it joins a network, obtains an IP address, opens a transport connection, and exchanges structured data with a remote server. The system built in this project follows a deliberately shorter path, and Table 5 identifies exactly which OSI layer is active at each stage of the Sender and Receiver design.

Table 4: OSI Layer Mapping of the Proposed System

OSI Layer	Standard Function	Role in the Proposed System
Physical Layer	Transmission of raw bits as electrical or radio signals	The 2.4 GHz Wi-Fi radio built into each ESP8266 chip generates and receives the radio waves that carry every ESP-NOW frame between the Sender and the Receiver.
Data Link Layer	Framing, addressing, and media access control	This is where ESP-NOW itself operates. It uses IEEE 802.11 vendor specific action frames and addresses the Receiver directly through its MAC address, without the association or handshake that ordinary Wi-Fi connections require.

Network Layer	Logical addressing and routing between networks	Not utilized. No IP address is assigned to either board, and no routing decision takes place, since both boards communicate directly at the data link layer alone.
Transport Layer	End to end reliability, segmentation, and flow control	Not utilized in the traditional sense. The send callback function used in the Sender code reports whether a frame was delivered, which resembles an acknowledgment, but this confirmation happens inside the data link layer frame exchange rather than through TCP or UDP.
Session Layer	Establishing, maintaining, and closing a communication session	Not applicable. The design is connectionless. Registering the Receiver as a peer only stores its MAC address; no session state is opened or kept alive between transmissions.
Presentation Layer	Data formatting, encoding, and translation between systems	The shared data structure containing the detection flags and the confirmed distance value is defined identically in both sketches, which is the manual equivalent of a presentation layer, ensuring both boards interpret the same bytes the same way.
Application Layer	Meaning and behavior given to the data by the end program	The sensor fusion rule, the one second confirmation timer, the three second cooldown, the LED status logic on both boards, and the LittleFS based logging on the Receiver together represent the application layer, since this is where raw sensor readings become a meaningful, confirmed alert.

Because the network, transport, and session layers are skipped entirely, the system avoids the processing time and code complexity that a full IP based stack would otherwise require. This is consistent with the field experiment study referenced in the literature review, which confirmed that ESP-NOW operates at the Wi-Fi data link layer using action frames, keeping its latency and energy consumption lower than protocols that depend on a complete network association process.

4.7 Justification for Selecting the ESP-NOW Protocol

ESP-NOW was selected as the communication protocol for this project for several concrete reasons rather than as a default choice. First, it directly answers the problem statement of this project, which is to remove dependency on a Wi-Fi router, an internet connection, or a cloud service, since ESP-NOW operates below the network layer and never requires a router or internet path. Second, it is built natively into the ESP8266 chip used on both boards, so no additional radio hardware or module needed to be purchased, matching the low cost design goal stated in the objectives. Third, the data exchanged between the Sender and the Receiver is very small, only a few detection flags and a single floating point distance value, and ESP-NOW is designed for exactly this kind of short, infrequent status packet rather than for large

continuous data streams. Fourth, the literature review conducted for this project cited a comparative performance study showing that ESP-NOW achieves a longer communication range than Bluetooth Low Energy while only doubling power consumption, and a separate outdoor field experiment confirming its low latency and low energy behavior compared to protocols that rely on full Wi-Fi association. These findings directly supported the choice of ESP-NOW for this project.

Several other wireless protocols could technically have been used to connect the Sender and the Receiver. Each one was considered and set aside for a specific, technical reason connected either to the OSI layers it would require or to the hardware it would need. Table 6 summarizes these alternatives.

Table 5: Alternative Wireless Protocols and Reasons They Were Not Selected

Alternative Protocol	OSI Layers It Would Involve	Reason It Was Not Selected
Bluetooth Low Energy (BLE)	Adds session layer style pairing and bonding, plus a service structure at the upper layers	Requires a pairing and connection process before any data can move, adding setup complexity for two boards that never change position. The comparative study cited in the literature review also found BLE has a shorter effective range than ESP-NOW at similar power consumption.
Standard Wi-Fi with TCP or IP (router based)	Requires the network layer for IP addressing and the transport layer for TCP or UDP, plus a physical router	Reintroduces exactly the dependency this project was designed to remove. A router failure or missing internet connection would stop the entire system from working, defeating the problem statement.
Zigbee	Adds a full mesh capable network layer with its own routing and addressing scheme	The ESP8266 has no built in Zigbee radio. A separate external transceiver module would be needed, increasing hardware cost and part count, which conflicts with the low cost design goal.
LoRa	Uses a different physical layer modulation scheme optimized for long range, low data rate links	Unnecessarily complex and low bandwidth for a short range, indoor, two node system, and requires a separate LoRa transceiver module not built into the ESP8266.
ESP-MESH	Extends the data link layer approach of ESP-NOW into multi hop, network layer style routing among many nodes	Designed for several nodes relaying data across a wider area, as seen in the residential monitoring study cited in the literature review. This project uses only two fixed nodes, so the added routing complexity is unnecessary now, although it remains a natural upgrade path already noted in the future work chapter.

4.8 Protocols Required If Cloud or Internet Connectivity Is Added

The future work chapter of this report mentions the possibility of adding mobile notifications or a hosted dashboard. If that extension were implemented, the system would need to make active use of the OSI layers that the current design intentionally skips. Table 7 outlines which protocol would come into use at each layer under such an extension.

Table 6: Protocols Required If Cloud or Internet Connectivity Were Added

OSI Layer	Protocol That Would Be Needed	Purpose
Network Layer	IP addressing over the local Wi-Fi network	Each ESP8266 would join the router as a station and obtain a local IP address, with the router performing network address translation to reach the internet.
Transport Layer	TCP, or UDP for lightweight telemetry	TCP would support reliable HTTP based communication, while UDP could be used for a lighter weight protocol such as CoAP if occasional data loss is acceptable.
Session Layer	Persistent broker connection (MQTT) or TLS session (HTTPS)	MQTT keeps a live connection open to a broker for continuous publishing, while HTTPS would involve a TLS handshake and session negotiation for secure requests.
Presentation Layer	JSON formatted data	Data would be represented as JSON rather than a raw C structure, since JSON is the format generally expected by cloud services and mobile applications.
Application Layer	MQTT, or HTTP REST calls to a cloud service	MQTT is a lightweight publish subscribe protocol suited to constrained IoT hardware and would replace the current raw ESP-NOW packet handling; HTTP REST calls to a service such as ThingSpeak or Firebase are a simpler alternative if a request and response style interaction is preferred.

This comparison highlights the deliberate trade off at the center of the current design. By remaining entirely within the physical, data link, presentation, and application layers, the project sacrifices remote internet access in exchange for independence from routers and cloud services, along with lower latency, lower power use, and simpler code, which directly matches the objectives and problem statement described earlier in this report.

5 Methodology

5.1 Working Principle

The complete system works in a continuous cycle rather than a one time action. Every loop cycle on the Sender board triggers a fresh ultrasonic reading using the standard trigger and echo timing method, and reads the current digital state of the PIR sensor. The ultrasonic distance is converted from the echo pulse duration into centimeters using the standard speed of sound calculation, and an object is only considered detected when the measured distance is 20 centimeters or less. In this step, the OSI model's Application layer is used, since reading the

sensors and deciding whether an object is within range is local application logic rather than network communication.

The Sender still lights its own three local LEDs using the original four state rule for on board indication, red when both sensors agree, yellow when only motion is present, green when only the ultrasonic sensor is present, and all three off when nothing is detected. This local indication is instant and is not affected by anything described below. In this step, the OSI model's Application layer is used, since lighting the local LEDs is application level output logic that does not involve the wireless link.

Wireless transmission, however, now follows a stricter two step rule. A message is only ever considered when both the ultrasonic sensor and the motion sensor detect something at the same time. The moment that happens, the Sender does not transmit immediately, it instead records the current distance as a baseline and starts a one second confirmation window. During that one second, it keeps re-checking the distance, and if the object drifts away from the baseline by more than a small tolerance, or if either sensor stops triggering, the confirmation is cancelled and nothing is sent. Only if the object stays at essentially the same distance for the full one second does the Sender build a small data packet, containing the detection flags and the confirmed distance, and transmit it to the Receiver using `esp_now_send`. In this step, the OSI model's Application layer is used for the confirmation decision, the Presentation layer is used to build the data packet, and the Data Link layer is used to carry out the `esp_now_send` transmission to the Receiver.

After a successful transmission, the Sender enters a three second cooldown period, during which it will not start a new confirmation cycle even if both sensors keep triggering. This prevents the same ongoing event from generating a new message every fraction of a second, and keeps the spacing between two messages at a minimum of about one second of confirmation plus three seconds of cooldown. In this step, the OSI model's Application layer is used, since the cooldown timer is a local timing control mechanism inside the Sender's application logic.

The Receiver board does not repeat any of this decision logic itself. It stays silent, showing only its green LED, until an ESP-NOW packet actually arrives. A receive callback function is triggered automatically by the ESP8266 core whenever a new packet arrives, and this callback copies the incoming data, records the exact time it was received, prints exactly one line describing the confirmed detection to the Serial Monitor, and appends that same line to a small text log file on the Receiver's own LittleFS flash file system, so a permanent, growing record of every confirmed detection is kept automatically without any extra hardware. In the main loop, the Receiver checks how long it has been since the last packet arrived, and if that gap exceeds four seconds, it falls back to the green LED, which is slightly longer than the Sender's own three second cooldown so the red LED does not flicker off during the normal gap between two confirmed messages of the same ongoing event. In this step, the OSI model's Data Link layer is used to receive the incoming frame, the Presentation layer is used to read the packet contents, and the Application layer is used to decide the LED state and write the log entry.

5.2 Flowchart

The flow chart below summarizes the Sender logic on the left side and the Receiver logic on the right side, including the confirmation and cooldown checks on the Sender, and the LittleFS logging step on the Receiver.

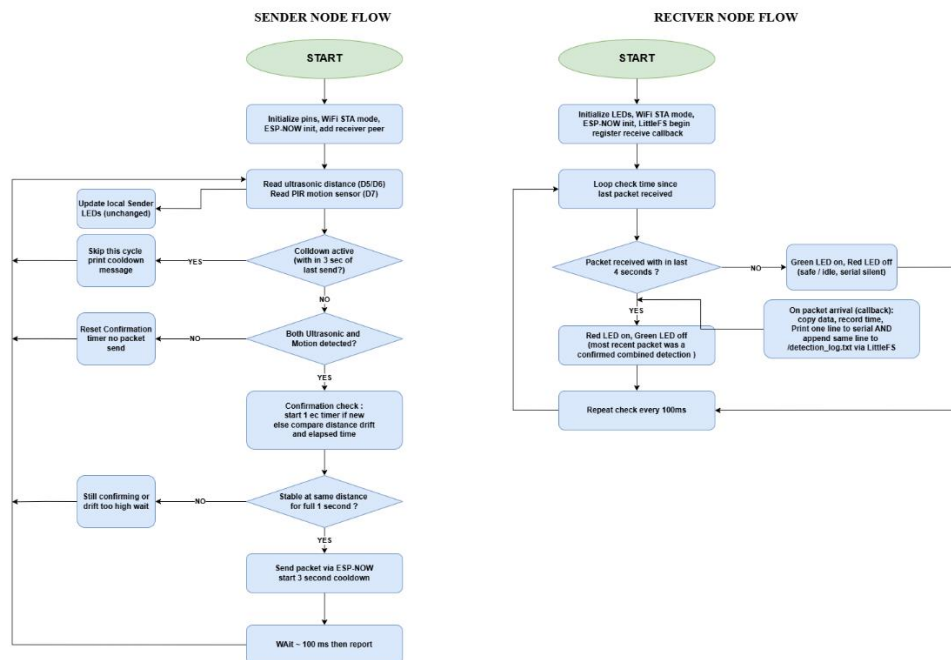


Figure 4: Combined working flow chart of the Sender node, including the confirmation and cooldown checks, and the Receiver node, including LittleFS logging

5.3 Algorithm

5.3.1 Sender Node Algorithm

- Start the Sender board and configure D1, D2, D3 as outputs for the LEDs, D5 as an output for the ultrasonic trigger pin, and D6 and D7 as inputs for the ultrasonic echo pin and the PIR sensor. In this step, the OSI model's Application layer is used, since configuring the LED and sensor pins is local device initialization that takes place outside the wireless communication stack.
- Switch the WiFi radio into station mode, print the board's own MAC address to the Serial Monitor for reference, and initialize the ESP-NOW protocol. In this step, the OSI model's Data Link layer is used, since switching to station mode and initializing ESP-NOW sets up the wireless data link stack that will later carry the frames.
- Register the Receiver's MAC address, 0xA4:0xCF:0x12:0xFF:0x88:0x31, as an ESP-NOW peer so that data can be sent to it later. In this step, the OSI model's Data Link layer is used, since registering the Receiver's MAC address as a peer is addressing performed at the data link layer.
- Inside the main loop, send a 10 microsecond HIGH pulse on the trigger pin and measure how long the echo pin stays HIGH afterward using the pulseIn function, then read the digital output of the PIR motion sensor. In this step, the OSI model's Application layer is

used, since reading the ultrasonic and PIR sensors is local data acquisition handled entirely by the application code.

- Update the three local Sender LEDs immediately using the four state rule, red for both sensors together, yellow for motion only, green for ultrasonic only, and all off for nothing detected. In this step, the OSI model's Application layer is used, since updating the local Sender LEDs is application level output logic that does not involve the wireless link.
- Check whether the three second post-send cooldown is still active. If it is, skip the rest of this cycle and do not start or continue any confirmation. In this step, the OSI model's Application layer is used, since checking the cooldown timer is a timing control decision made entirely within the application code.
- If both the ultrasonic sensor and the motion sensor are triggered at the same time and no confirmation is currently running, record the current distance as a baseline and start a one second confirmation timer. In this step, the OSI model's Application layer is used, since this is a local decision made by the application logic based on the sensor readings and the running timer.
- If a confirmation is already running, compare the current distance with the recorded baseline. If it has drifted by more than the allowed tolerance, cancel the confirmation and wait for a fresh detection. In this step, the OSI model's Application layer is used, since this is a local decision made by the application logic based on the sensor readings and the running timer.
- If the confirmation timer reaches one full second while the distance has stayed within tolerance, treat the detection as confirmed. In this step, the OSI model's Application layer is used, since this is a local decision made by the application logic based on the sensor readings and the running timer.
- Once confirmed, build a data packet containing the detection flags and the confirmed distance, transmit it to the Receiver's MAC address using `esp_now_send`, and start the three second cooldown timer. In this step, the OSI model's Presentation layer is used to build the data packet, and the Data Link layer is used to transmit it through `esp_now_send`.
- If either sensor stops triggering before the one second confirmation completes, cancel the confirmation and report that no packet was sent for this cycle. In this step, the OSI model's Application layer is used, since this is a local decision made by the application logic based on the sensor readings and the running timer.
- Print the current sensor readings, confirmation progress, or cooldown status to the Serial Monitor for debugging, wait briefly, and repeat the loop indefinitely. In this step, the OSI model's Application layer is used, since printing debug information to the Serial Monitor is local diagnostic output with no role in network communication.

5.3.2 Receiver Node Algorithm

- Start the Receiver board and configure D1 and D2 as outputs for the green LED and the red LED. In this step, the OSI model's Application layer is used, since configuring the Receiver's LED pins is local device initialization that takes place outside the wireless communication stack.

- Switch the WiFi radio into station mode, initialize the ESP-NOW protocol, and start the LittleFS flash file system used for the notepad log file. In this step, the OSI model's Data Link layer is used for the Wi-Fi station mode and ESP-NOW initialization, while starting the LittleFS file system is a local storage operation that falls outside the OSI communication stack.
- Register a receive callback function so that the ESP8266 core automatically calls it whenever a new ESP-NOW packet arrives from the Sender. In this step, the OSI model's Data Link layer is used, since registering the receive callback prepares the board to accept incoming ESP-NOW frames at the data link layer.
- Inside the receive callback, copy the incoming packet into a local data structure, record the current time as the last time data was received, and build one line of text describing the confirmed detection and its distance. In this step, the OSI model's Presentation layer is used, since copying the incoming packet into a local data structure is where the received bytes are interpreted into meaningful values.
- Print that line to the Serial Monitor, then open the notepad log file in append mode, write the same line into it, and close the file, so the Serial Monitor and the log file always match exactly. In this step, the OSI model's Application layer is used, since printing to the Serial Monitor and writing to the log file are local diagnostic and storage actions handled by the application code.
- Inside the main loop, calculate how much time has passed since the last packet was received. In this step, the OSI model's Application layer is used, since calculating the elapsed time since the last packet is a timing control decision made within the application code.
- If more than four seconds have passed since the last packet, or if no packet has ever been received, turn the green LED on and the red LED off, and print nothing new to the Serial Monitor. In this step, the OSI model's Application layer is used, since deciding the LED color based on the received status is application level output logic.
- Otherwise, meaning a recent packet reported a confirmed combined detection, turn the red LED on and the green LED off. In this step, the OSI model's Application layer is used, since deciding the LED color based on the received status is application level output logic.
- Repeat this check continuously, roughly every 100 milliseconds, so the Receiver LEDs always reflect the latest known status of the Sender, while the Serial Monitor stays silent between detections.

5.4 Functionalities

Dual sensor fusion: the Sender only treats an event as significant when both the ultrasonic distance reading and the PIR motion reading agree at the same time, which is far more reliable than trusting either sensor alone.

Confirmation before transmission: a one second stability check, comparing the live distance against a recorded baseline, filters out brief, accidental, or noisy triggers before anything is ever sent wirelessly.

Cooldown controlled spacing: a three second cooldown after every confirmed transmission keeps the Receiver from being flooded with repeated messages while the same object or person is still present.

Router free wireless transmission: detection status travels from the Sender to the Receiver using ESP-NOW, so the system keeps working even if no WiFi router or internet connection is present anywhere nearby.

Silent by default Receiver: the Receiver's Serial Monitor stays completely quiet until a confirmed detection actually arrives, which makes the output easy to read during a live demonstration.

Automatic notepad logging: every confirmed detection line is appended to a text log file on the Receiver's own LittleFS flash storage, building a permanent record with no additional hardware or software.

Automatic link loss detection: the Receiver independently tracks how long it has been since the last packet arrived, and falls back to a safe green state if the Sender stops responding, rather than freezing on the last known status.

6 Results and Discussion

The system was tested by manually creating different sensor conditions in front of the Sender board and observing how the Sender LEDs, the Receiver LEDs, and both Serial Monitors responded, including the new one second confirmation window and the three second cooldown. The table below lists the test cases that were used and the outcome that was actually observed on the hardware during testing.

Table 7: Test cases and observed hardware results

Test Case	Condition Created	Sender LED Observed	Receiver LED Observed
TC1	No Object nearby, no movement	All LEDs off	Green LED on, Serial silent
TC2	Object placed with in 20cm only no movement	Green LED on	Stays Green LED on, no packet sent
TC3	Motion only no object with in 20cm	Yellow LED on	Stays Green LED on no packet sent
TC4	Both triggered but object pulled away before 1 second	Red LED on briefly	Confirmation cancelled stays green
TC5	Both triggered and held steady for a full 1 second	Red LED on	Red LED on after confirmation one line logged

TC6	Object and motion held continuously for 10+ seconds	Red LED stays on	Red LED stays on new log line about every 4 seconds
TC7	Sender powered off or moved out of range	Not applicable	Green LED on after 4 second timeout

The photographs and Serial Monitor captures below were taken directly from the working hardware and support the test results summarized above. The three LED colour photographs show the physical states that the system reaches once a condition has fully played out, and the two Serial Monitor captures show the underlying detection and packet activity.

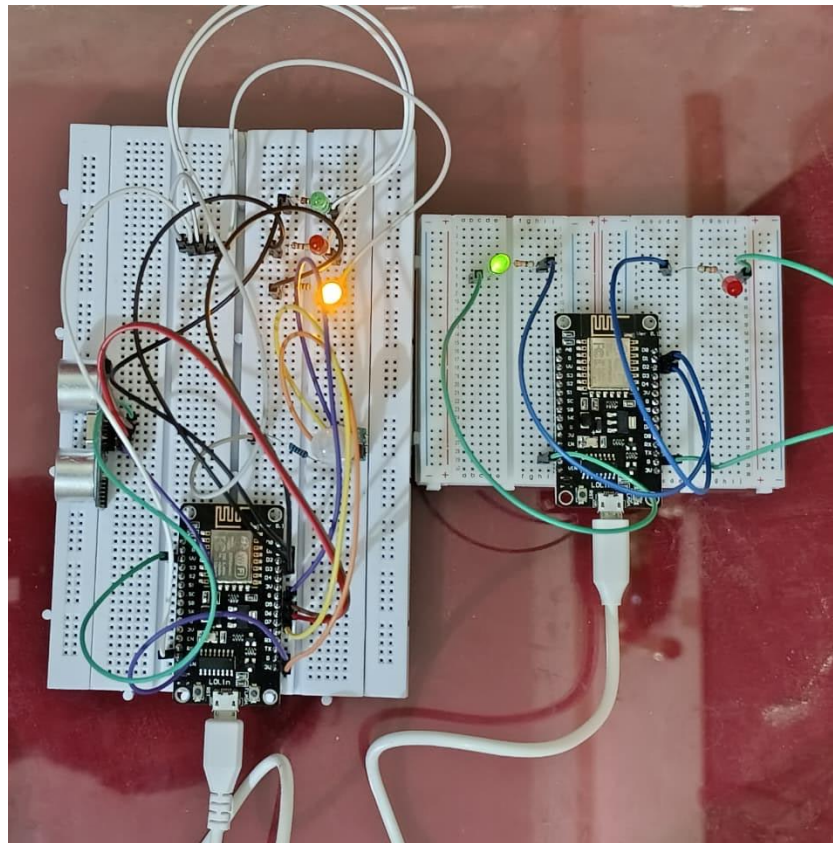


Figure 5: Both nodes running together, the Sender showing its local yellow LED for a motion only trigger, while the Receiver stays on green since a single sensor trigger is never transmitted

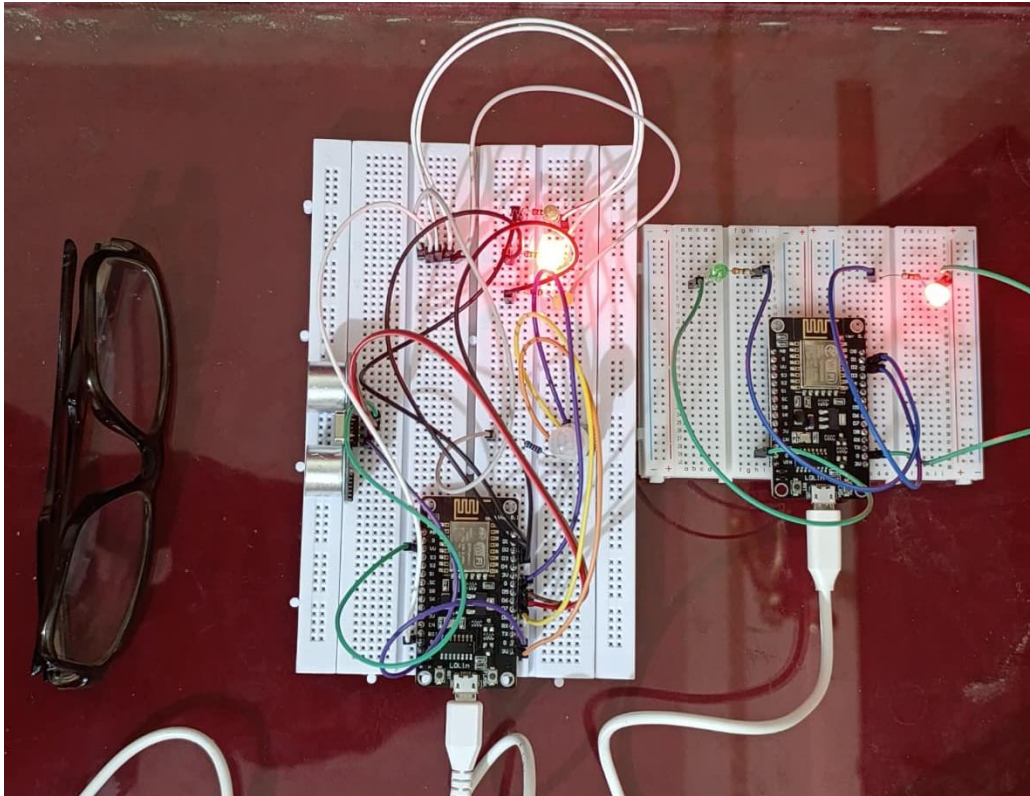


Figure 6: Both nodes running together after a combined detection passed the one second confirmation window, showing the Sender red LED and the Receiver red LED together

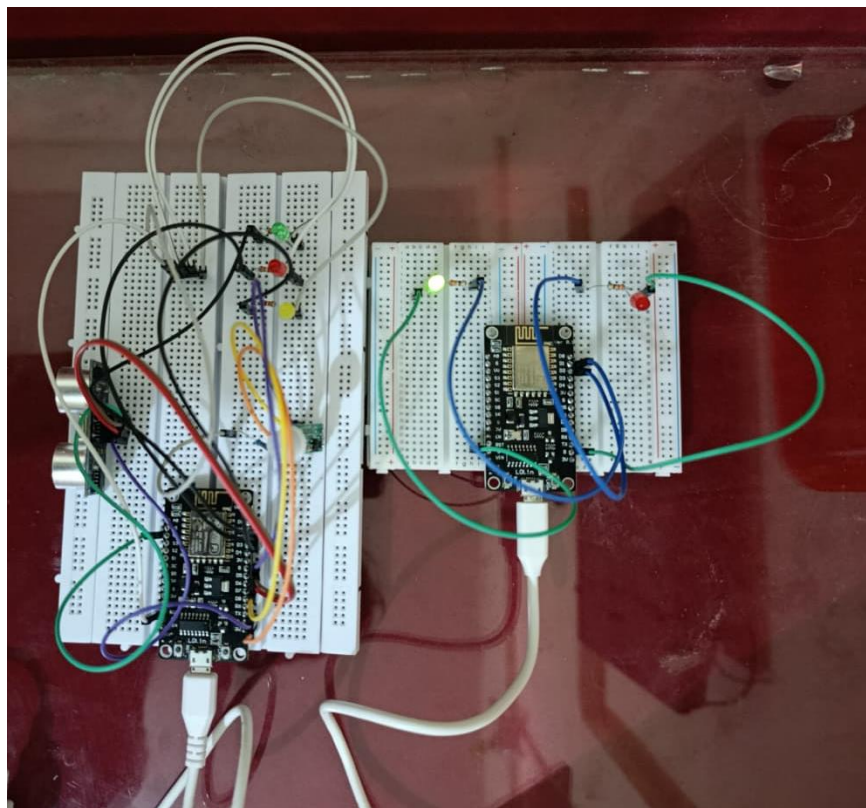


Figure 7: Both nodes running together with the Receiver back on green, the default safe state, while the Sender LEDs react locally to brief sensor activity that never completed a full confirmation

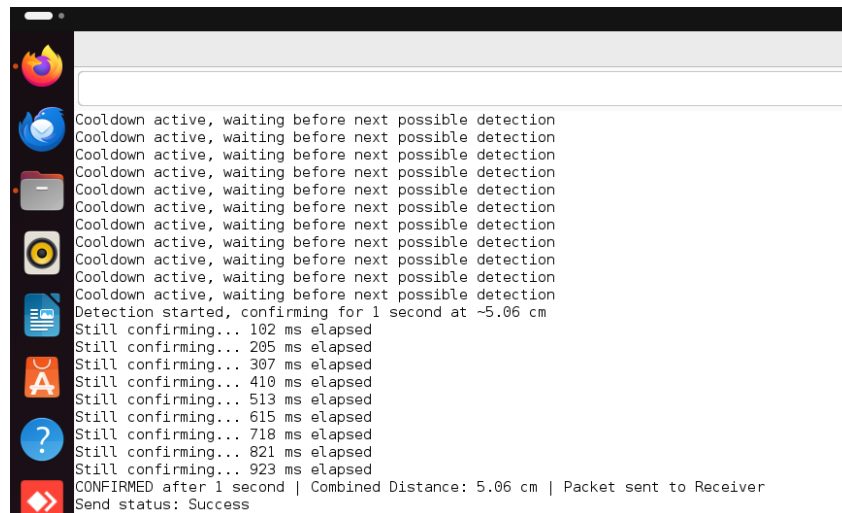


Figure 8: Sender Serial Monitor output showing the cooldown message while waiting, then a fresh detection starting the one second confirmation countdown, and finally the *CONFIRMED* line once the object stayed at the same distance for the full second, followed by a successful send status



Figure 9: Receiver Serial Monitor output showing one line printed per confirmed combined detection, each with its own time stamp and combined distance, spaced several seconds apart by the Sender's cooldown rather than printing continuously

The captured results confirm that the updated decision logic described in Chapter 4 behaved exactly as designed. During TC2 and TC3, only one sensor was active, the Sender showed its own local yellow or green LED, but no packet was ever transmitted, and the Receiver correctly stayed on green throughout, with nothing printed on its Serial Monitor, confirming that single sensor triggers no longer reach the Receiver at all.

During TC4, briefly pulling the object away before the one second confirmation window completed correctly cancelled the pending transmission, and the Receiver never showed red for that attempt, which confirms that the confirmation step successfully filters out short lived or accidental triggers.

During TC5, holding both sensors triggered for the full one second produced exactly one transmitted packet, after which the Receiver's red LED turned on, one line appeared on its

Serial Monitor, and the same line was found afterwards inside /detection_log.txt using the separate log reader sketch, confirming that the Serial Monitor and the notepad log file always match.

During TC6, keeping the object and motion present across repeated trials showed the three second Sender side cooldown in action, with a new confirmed message and log line appearing roughly every four to seven seconds in the captured Receiver Serial Monitor output, rather than multiple times per second as in the original design, while the Receiver's slightly longer four second timeout kept the red LED steady across these gaps instead of flickering back to green.

Overall, the combination of confirmation and cooldown reduced the message rate from roughly two packets every second in the original design to at most one confirmed packet every few seconds, while still reliably reporting every genuine, sustained combined detection.

7 Conclusion

7.1 Project Summary

This project successfully demonstrates a low cost, internet free, wireless detection and alert system using two ESP8266 NodeMCU boards and the ESP-NOW protocol. By combining an ultrasonic sensor and a motion sensor on the Sender side, and only transmitting after a one second confirmation of a combined detection, the system produces a far more reliable and controlled alert path than the original design, while a three second cooldown and automatic LittleFS logging on the Receiver round out a practical, self contained monitoring solution.

7.2 Achievements

The project achieved a fully working Sender and Receiver pair, verified pin level wiring for every sensor and LED, a tested confirmation and cooldown state machine that filters out short lived triggers and limits message rate, a Receiver that stays silent until a genuine detection is confirmed, and an automatic notepad style log file that always matches what is printed on the Serial Monitor. All of this was verified directly on hardware using LED observation, Serial Monitor logs, and the log file itself, rather than simulation alone.

7.3 Advantages

The system does not depend on any WiFi router, internet connection, or cloud subscription, which removes recurring costs and also removes the risk of the system failing simply because an internet connection is down.

Combining two different sensor types, and only acting once both agree for a full second, reduces false alarms far more effectively than the original single check design, since a brief or accidental trigger is no longer enough to produce an alert.

The three second cooldown keeps the wireless channel and the Receiver's log file from being flooded with near duplicate messages while an object or person remains in place.

The automatic LittleFS logging gives the project a permanent, self contained record of every confirmed detection without needing an SD card, a computer, or any internet service running at the same time.

7.4 Limitations

ESP-NOW communication range on the ESP8266 is limited, and depending on the physical environment, walls, and interference, the practical range is usually smaller than open field WiFi range, so the Sender and Receiver need to stay within a reasonable distance of each other.

The current design only supports one Sender talking to one Receiver. If a second Sender board needed to be added, the Receiver code would need to be modified to identify which Sender a packet came from.

The ultrasonic detection threshold, the one second confirmation window, and the three second cooldown are all fixed constants in the code. Any change to these values currently requires editing and re uploading the sketches rather than adjusting them from an external interface.

The log file currently uses millis() based time stamps, which reset every time the Receiver reboots and do not represent real world clock time, so the notepad log is only meaningful for relative timing within a single power cycle.

8 Future Work

A buzzer or a mobile notification gateway could be added on the Receiver side, so that an alert is not only visual but also audible or pushed to a phone, which would make the system useful even when nobody is looking directly at the LEDs.

An OLED display could be added to either board to show the live distance value and a short text description of the current status, instead of relying only on LED colors.

The Receiver code could be extended to recognize multiple Sender boards at the same time by including a unique identifier inside each data packet, allowing one Receiver to monitor several locations at once.

A real time clock module, or an NTP time sync whenever WiFi is briefly available, could replace the millis() based time stamps in the log file with genuine calendar dates and times.

The fixed confirmation window, cooldown duration, and distance tolerance could be made configurable at runtime, for example through a small web page hosted on the Receiver, instead of being fixed constants in the code.

For outdoor or long term deployment, the Sender could be powered using a battery pack with a small solar panel, and the ESP8266 deep sleep mode could be used between readings to significantly extend battery life.

9 References

[1] Y. Bai et al., "Use of Ultrasonic Signal Coding and PIR Sensors to Enhance the Sensing Reliability of an Embedded Surveillance System," 2013. [Online]. Available: <https://www.researchgate.net/publication/261082178>

- [2] "IoT for Residential Monitoring Using ESP8266 and ESP-NOW Protocol," ResearchGate. [Online]. Available: <https://www.researchgate.net/publication/363492054>.
- [3] "An Efficient Networking Solution for Extending and Controlling Wireless Sensor Networks Using Low-Energy Technologies," *IEEE Access*, 2021. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8627223/>
- [4] "ESP-NOW Performance in Outdoor Environments: Field Experiments and Analysis," *IFIP Wireless On-demand Network Systems and Services (WONS)*, 2025. [Online]. Available: <https://dl.ifip.org/db/conf/wons/wons2025/1571077625.pdf>
- [5] Espressif Systems and the ESP8266 Arduino Core Contributors, "espnw.h Header File Defining the ESP-NOW API Functions for ESP8266," GitHub Repository. [Online]. Available: <https://github.com/esp8266/Arduino/blob/master/tools/sdk/include/espnw.h>.
- [6] Arduino Documentation, "Device-to-Device Communication with ESP-NOW." [Online]. Available: <https://docs.arduino.cc/tutorials/nano-esp32/esp-now/>.
- [7] Random Nerd Tutorials, "Getting Started with ESP-NOW (ESP8266 NodeMCU with Arduino IDE)." [Online]. Available: <https://randomnerdtutorials.com/esp-now-esp8266-nodemcu-arduino-ide/>.
- [8] ElectronicWings, "Beginner's Guide: ESP-NOW ESP8266 (NodeMCU) Using Arduino IDE." [Online]. Available: <https://www.electronicwings.com/nodemcu/beginners-guide-esp-now-esp8266-nodemcu-using-arduino-ide>.
- [9] K. Aditya, "ESP8266 PIR Home Security and Notifier," Hackster.io. [Online]. Available: <https://www.hackster.io/kumaraditya303/esp8266-pir-home-security-notifier-b8245c>.
- [10] Random Nerd Tutorials, "Complete Guide for Ultrasonic Sensor HC-SR04 with Arduino." [Online]. Available: <https://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/>.
- [11] HandsOn Technology, "HC-SR04 Ultrasonic Sensor Module User Guide." [Online]. Available: <https://www.handsontec.com/dataspecs/HC-SR04-Ultrasonic.pdf>.
- [12] Components101, "HC-SR04 Ultrasonic Sensor: Working, Pinout, Features and Datasheet." [Online]. Available: <https://components101.com/sensors/ultrasonic-sensor-working-pinout-datasheet>.

10 Appendix

10.1 Sender.ino

```
SENDER NODEMCU (ESP8266)
```

```
MAC Address of this device: 8C:AA:B5:52:BE:F4
```

```
Sends detection status to Receiver MAC: A4:CF:12:FF:88:31
```

Pin mapping used in this code:

D1 Green LED (through 220 ohm resistor)

D2 Red LED (through 220 ohm resistor)

D3 Yellow LED (through 220 ohm resistor)

D5 Ultrasonic HC-SR04 TRIG

D6 Ultrasonic HC-SR04 ECHO

D7 PIR Motion Sensor OUT

G Common ground for LED cathodes, ultrasonic GND, PIR GND

VU Ultrasonic VCC (5V line)

3V PIR VCC

Local LED logic (unchanged, for on-board indication only):

Both ultrasonic and motion detect something -> Red LED ON

Only motion detects something -> Yellow LED ON

Only ultrasonic detects something -> Green LED ON

Nothing detected -> All LEDs OFF

WIRELESS RULE - PART 1 (unchanged):

A message is only ever considered when BOTH the ultrasonic sensor
AND the PIR motion sensor detect something AT THE SAME TIME.

If only one sensor detects something, or if nothing is detected,
no message goes anywhere near the Receiver.

WIRELESS RULE - PART 2 (new, confirmation + cooldown):

Instead of sending the moment both sensors trigger, the Sender now
waits and double-checks first, so a single quick, noisy, or
accidental trigger cannot cause a message to be sent.

Step A - CONFIRMATION WINDOW (about 1 second):

The instant both sensors trigger together, the Sender notes the current ultrasonic distance as a baseline and starts a 1 second timer. During that 1 second, it keeps re-checking the ultrasonic distance. If the object stays within a small tolerance of that same baseline distance, and both sensors remain triggered, for the full 1 second, the detection is treated as CONFIRMED.

If the object moves away, the distance changes too much, or either sensor stops triggering before the 1 second is complete, the confirmation is cancelled and the Sender simply goes back to watching for a fresh detection, nothing is sent.

Step B - COOLDOWN WINDOW (3 seconds):

Once a confirmed detection is actually sent to the Receiver, the Sender will not start a new confirmation cycle for the next 3 seconds, even if both sensors keep triggering. This stops the same ongoing event from flooding the Receiver with many messages per second, and matches the requested minimum spacing of 1 to 3 seconds between messages.

Detection distance threshold for ultrasonic: 20 cm

Data is sent to the receiver using ESP-NOW, no WiFi router or internet involved.

*/

```
#include <ESP8266WiFi.h>
```

```
#include <math.h>
```

```
extern "C" {
```

```
    #include <espnow.h>
```

```
}
```

```

#define GREEN_LED D1

#define RED_LED D2

#define YELLOW_LED D3

#define TRIG_PIN D5

#define ECHO_PIN D6

#define PIR_PIN D7


#define DETECTION_DISTANCE_CM 20


/* Tolerance used while confirming that the object is still at the
   same distance. A small allowance handles normal sensor jitter. */

#define DISTANCE_TOLERANCE_CM 3.0


/* How long the detection must stay stable before it is trusted */

const unsigned long CONFIRM_DURATION_MS = 1000;


/* Minimum gap enforced between two messages sent to the Receiver */

const unsigned long COOLDOWN_DURATION_MS = 3000;


uint8_t receiverMac[] = {0xA4, 0xCF, 0x12, 0xFF, 0x88, 0x31};


typedef struct struct_message {

    bool anyDetected;

    bool motionDetected;

    bool ultrasonicDetected;

    float distance;

} struct_message;


struct_message outgoingData;

```

```

unsigned long lastSendTime = 0;    /* used for the 3 second cooldown */

bool cooldownActive = false;

bool confirming = false;          /* true while inside the 1 second check */

unsigned long confirmStartTime = 0;

float confirmBaselineDistance = 0;

void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {

    Serial.print("Send status: ");

    Serial.println(sendStatus == 0 ? "Success" : "Failed");

}

float readDistanceCM() {

    digitalWrite(TRIG_PIN, LOW);

    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);

    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);

    if (duration == 0) {

        return -1;

    }

    float distanceCM = duration * 0.0343 / 2.0;

    return distanceCM;

}

void setup() {

```

```

Serial.begin(115200);

pinMode(GREEN_LED, OUTPUT);

pinMode(RED_LED, OUTPUT);

pinMode(YELLOW_LED, OUTPUT);

pinMode(TRIG_PIN, OUTPUT);

pinMode(ECHO_PIN, INPUT);

pinMode(PIR_PIN, INPUT);


digitalWrite(GREEN_LED, LOW);

digitalWrite(RED_LED, LOW);

digitalWrite(YELLOW_LED, LOW);


WiFi.mode(WIFI_STA);

Serial.print("Sender MAC Address: ");

Serial.println(WiFi.macAddress());


if (esp_now_init() != 0) {

    Serial.println("ESP-NOW init failed, restarting");

    ESP.restart();

}


esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);

esp_now_add_peer(receiverMac, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);

esp_now_register_send_cb(OnDataSent);

}


void loop() {

    float distance = readDistanceCM();

```

```

bool ultrasonicDetected = (distance > 0 && distance <= DETECTION_DISTANCE_CM);

bool motionDetected = (digitalRead(PIR_PIN) == HIGH);

bool bothDetected = ultrasonicDetected && motionDetected;


/* Local Sender LEDs still react immediately, this part is unchanged */

digitalWrite(GREEN_LED, LOW);

digitalWrite(RED_LED, LOW);

digitalWrite(YELLOW_LED, LOW);


if (bothDetected) {

    digitalWrite(RED_LED, HIGH);

} else if (motionDetected) {

    digitalWrite(YELLOW_LED, HIGH);

} else if (ultrasonicDetected) {

    digitalWrite(GREEN_LED, HIGH);

}


/* Check whether the 3 second cooldown after the last send is still active */

if (cooldownActive && (millis() - lastSendTime >= COOLDOWN_DURATION_MS)) {

    cooldownActive = false;

}


if (cooldownActive) {

    /* Still inside the post-send cooldown window, ignore everything */

    Serial.println("Cooldown active, waiting before next possible detection");

}

else if (bothDetected) {

    if (!confirming) {

        /* A fresh detection just started, begin the 1 second confirmation */

```

```

confirming = true;

confirmStartTime = millis();

confirmBaselineDistance = distance;

Serial.print("Detection started, confirming for 1 second at ~");

Serial.print(confirmBaselineDistance);

Serial.println(" cm");

} else {

float drift = fabs(distance - confirmBaselineDistance);

if (drift > DISTANCE_TOLERANCE_CM) {

/* Object moved too much, this was not a stable detection */

Serial.println("Distance changed too much, confirmation cancelled");

confirming = false;

} else if (millis() - confirmStartTime >= CONFIRM_DURATION_MS) {

/* Stayed steady for the full 1 second, this is a confirmed detection */

outgoingData.anyDetected = true;

outgoingData.motionDetected = true;

outgoingData.ultrasonicDetected = true;

outgoingData.distance = distance;

esp_now_send(receiverMac, (uint8_t *) &outgoingData, sizeof(outgoingData));

Serial.print("CONFIRMED after 1 second | Combined Distance: ");

Serial.print(distance);

Serial.println(" cm | Packet sent to Receiver");

confirming = false;

cooldownActive = true;

lastSendTime = millis();

```

```

    } else {

        Serial.print("Still confirming... ");

        Serial.print((millis() - confirmStartTime));

        Serial.println(" ms elapsed");

    }

}

}

else {

    /* Either sensor stopped triggering, or neither ever triggered */

    if (confirming) {

        Serial.println("Detection lost before confirmation completed, cancelled");

    }

    confirming = false;

    Serial.print("Ultrasonic: ");

    Serial.print(ultrasonicDetected ? "DETECTED" : "clear");

    Serial.print(" | Distance: ");

    Serial.print(distance);

    Serial.print(" cm | Motion: ");

    Serial.print(motionDetected ? "DETECTED" : "clear");

    Serial.println(" | No packet sent (single or no detection)");

}

delay(100);

}

```

10.2 Receiver.ino

```

/*

RECEIVER NODEMCU (ESP8266)

```

MAC Address of this device: A4:CF:12:FF:88:31

Receives detection status from Sender MAC: 8C:AA:B5:52:BE:F4

Pin mapping used in this code:

D1 Green LED (through 220 ohm resistor)

D2 Red LED (through 220 ohm resistor)

G Common ground for LED cathodes

UPDATED LOGIC (as instructed by supervisor):

The Sender now only transmits a packet when BOTH the ultrasonic sensor AND the PIR motion sensor detect something at the same time.

So on the Receiver side:

A packet arrives -> Red LED ON, print the combined

detection distance to Serial Monitor

No packet arrives / only

one sensor was triggered on

the Sender / nothing at all

was detected -> Green LED ON (safe / idle state)

NOTEPAD LOGGING (new):

Every line that is printed on the Serial Monitor for a received detection is ALSO appended, at the same time, to a small text log file stored on the Receiver's own flash memory using LittleFS.

The file is called /detection_log.txt and keeps growing with one line per detection event, acting like a running notepad file.

How to read the log file later:

Option 1: Add a small extra sketch that calls LittleFS.begin(),

opens "/detection_log.txt" in read mode, and prints its full content to the Serial Monitor.

Option 2: Use the "ESP8266 LittleFS Filesystem Uploader / Downloader" tool in the Arduino IDE to pull the file to your computer.

If the connection with the sender is lost (no packet within the timeout window) the receiver falls back to Green LED, matching the same behavior as never having received a combined detection.

```
*/

#include <ESP8266WiFi.h>

#include <LittleFS.h>

extern "C" {

    #include <espnow.h>

}

#define GREEN_LED D1

#define RED_LED D2

/* Slightly longer than the Sender's 3 second cooldown, so the red
   LED does not flicker back to green during the normal gap between
   two confirmed messages while the object is still there. */

const unsigned long noDataTimeout = 4000;

const char *LOG_FILE_PATH = "/detection_log.txt";

typedef struct struct_message {

    bool anyDetected;

    bool motionDetected;

    bool ultrasonicDetected;
```

```

float distance;

} struct_message;

struct_message incomingData;

unsigned long lastReceiveTime = 0;

bool hasReceivedOnce = false;

/* Appends one line of text to the notepad log file on LittleFS */

void logToFile(const String &line) {

    File f = LittleFS.open(LOG_FILE_PATH, "a");

    if (!f) {

        Serial.println("Warning: could not open log file for writing");

        return;

    }

    f.println(line);

    f.close();

}

void OnDataRecv(uint8_t *mac, uint8_t *data, uint8_t len) {

    memcpy(&incomingData, data, sizeof(incomingData));

    lastReceiveTime = millis();

    hasReceivedOnce = true;

    /* Build one line of text that goes to BOTH the Serial Monitor

    and the notepad log file, so they always match exactly. */

    String line = "[" + String(millis() / 1000) + "s] BOTH sensors detected on Sender | Combined Distance: " +

        String(incomingData.distance) + " cm | Receiver LED: RED";

    Serial.println(line);

```

```

logToFile(line);
}

void setup() {

  Serial.begin(115200);

  pinMode(GREEN_LED, OUTPUT);

  pinMode(RED_LED, OUTPUT);

  digitalWrite(GREEN_LED, HIGH);

  digitalWrite(RED_LED, LOW);

  WiFi.mode(WIFI_STA);

  /* Start the on-chip file system used for the notepad log file */

  if (!LittleFS.begin()) {

    LittleFS.format();

    LittleFS.begin();

  }

  if (esp_now_init() != 0) {

    ESP.restart();

  }

  esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);

  esp_now_register_recv_cb(OnDataRecv);

  /* Nothing else is printed here on purpose. The Serial Monitor

  stays silent until the Sender actually reports that BOTH

  sensors detected something together. */

```

```

}

void loop() {

  bool connectionLost = (millis() - lastReceiveTime > noDataTimeout);

  if (!hasReceivedOnce || connectionLost || !incomingData.anyDetected) {

    digitalWrite(GREEN_LED, HIGH);

    digitalWrite(RED_LED, LOW);

  } else {

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(RED_LED, HIGH);

  }

  delay(100);
}

```

10.3 read_log.ino

This sketch is not part of normal operation. It is uploaded temporarily, only when the student wants to retrieve the content of /detection_log.txt from the Receiver's LittleFS storage, print it on the Serial Monitor, and copy it into a text file on a computer.

```

/*
LOG FILE READER (ESP8266 NodeMCU)

Purpose:
This is a small, separate sketch used ONLY to open the notepad
log file that the Receiver sketch has been writing to, and print
its full content on the Serial Monitor so you can copy it out to
a real .txt file on your computer.

IMPORTANT - Before uploading this sketch:
Go to Tools > Erase Flash, and make sure it is set to "Only Sketch"
(NOT "All Flash Contents"). This setting keeps your LittleFS log
file safe while a new sketch is uploaded. If it is set to erase
all flash contents, your log file will be permanently deleted
the moment you upload.

How to use:
1. Upload this sketch to the Receiver NodeMCU (same board, temporarily).
2. Open Tools > Serial Monitor, set baud rate to 115200.
3. The full content of detection_log.txt will be printed once.
4. Select all the printed text, copy it, and paste it into a real

```

Notepad file on your computer, then save it as detection_log.txt.
5. When you are done reading, re-upload the original receiver.ino sketch to go back to normal operation. The log file and its content will still be there, since it lives in LittleFS, not in the sketch itself.

```
*/  
  
#include <LittleFS.h>  
  
const char *LOG_FILE_PATH = "/detection_log.txt";  
  
void setup() {  
  Serial.begin(115200);  
  delay(1000);  
  
  Serial.println("=====");  
  Serial.println(" Reading LittleFS Log File");  
  Serial.println("=====");  
  
  if (!LittleFS.begin()) {  
    Serial.println("LittleFS mount failed. No log file available.");  
    return;  
  }  
  
  if (!LittleFS.exists(LOG_FILE_PATH)) {  
    Serial.println("No log file found yet. Nothing has been detected so far.");  
    return;  
  }  
  
  File f = LittleFS.open(LOG_FILE_PATH, "r");  
  if (!f) {  
    Serial.println("Could not open the log file.");  
    return;  
  }  
  
  Serial.print("File size: ");  
  Serial.print(f.size());  
  Serial.println(" bytes");  
  Serial.println("-----");  
  
  while (f.available()) {  
    Serial.write(f.read());  
  }  
  
  f.close();  
  
  Serial.println("-----");  
  Serial.println("End of log file. Copy everything above");  
  Serial.println("this line (from File size onward) into");  
  Serial.println("a Notepad file on your computer.");  
}  
  
void loop() {  
  /* Nothing to repeat, this sketch only runs once at startup */  
}
```